

Smart Healthcare Diagnosis API

Complete Viva Voce & Academic Defense Master Guide

Project Title: Smart Healthcare Diagnosis API	Domain: Machine Learning & Healthcare Microservices
Candidate Name: Krish Gaikwad	Tech Stack: Python 3.12, FastAPI, Scikit-Learn, Pydantic v2
Core Goal: Real-time Disease Diagnosis & Risk Stratification	Repository: github.com/krishgaikwad42-hood/apiproject-

Examiner Perspective Note: In viva examinations, professors evaluate two fundamental domains: **(1) Mathematical & Data Science rigor** (handling disguised zeros, data leakage prevention, algorithm benchmarking, precision/recall trade-offs) and **(2) Software Engineering & Production readiness** (FastAPI asynchronous event loops, Pydantic schemas, correlation ID tracing, latency SLAs, and automated test coverage). This document prepares you to master both.

1. Project Architecture & End-to-End Workflow

The system is an end-to-end clinical machine learning service designed to provide sub-50ms diagnostic predictions. The complete request lifecycle flows through 5 distinct architectural layers:

- **Layer 1 — Client Interface:** Web Dashboard UI, Postman, or third-party EHR systems submit HTTP POST JSON payloads.
- **Layer 2 — Observability & Tracing Middleware:** Intercepts incoming requests, generates a unique X-Request-ID UUID4 for distributed correlation tracing, measures execution latency in milliseconds, and logs client IP.
- **Layer 3 — Pydantic Validation & Domain Integrity:** Strict type verification (Float, Integer). Enforces clinical bounds (e.g., Glucose 40–500 mg/dL, Age 1–120) and dual-gender constraints (Male patients cannot have pregnancies > 0).
- **Layer 4 — Serialized Preprocessing Pipeline:** Pre-fitted SimpleImputer(strategy='median') resolves disguised zeros followed by feature normalization using RobustScaler.
- **Layer 5 — In-Memory ML Inference Engine:** Model generates risk class (Diabetic / Non-Diabetic), calibrated prediction probability (0.0 to 1.0), confidence percentage, and stratified risk tier (Low <35%, Moderate 35–65%, High >65%).

2. Dataset & The Critical 'Disguised Zeros' Finding

The model was trained on the **Pima Indians Diabetes Dataset** (National Institute of Diabetes and Digestive and Kidney Diseases).

Metric / Parameter	Clinical Detail / Statistical Property
Total Dataset Size	768 Patient Records, 8 Clinical Feature Dimensions + 1 Target (Outcome)
Target Class Balance	65.1% Negative (500 non-diabetic) vs. 34.9% Positive (268 diabetic) — Moderately Imbalanced
Disguised Missing Values	Numerical zeros entered in place of missing tests: Glucose (5), Blood Pressure (35), Skin Thickness (227), Insulin (374), BMI (11).
Imputation Strategy	Zero values in physiological fields converted to NaN and imputed with Median (less sensitive to skew/outliers than Mean).
Data Leakage Defense	Train/Test split (80/20 Stratified) performed BEFORE fitting Imputer and Scaler. Test data is never observed during transformation fitting.

3. Machine Learning Benchmark & Model Selection

Six supervised learning classification algorithms were rigorously benchmarked with stratified cross-validation. In clinical systems, **Recall (Sensitivity)** is prioritized over raw accuracy because a False Negative (missed diabetic patient) is clinically catastrophic.

Algorithm	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Decision Tree	74.7%	63.0%	72.2%	67.2%	0.741
Random Forest	76.6%	68.8%	61.1%	64.7%	0.835
Logistic Regression	77.9%	72.7%	59.3%	65.3%	0.842
Support Vector Machine (SVM)	76.0%	70.0%	51.9%	59.6%	0.814
K-Nearest Neighbors (KNN)	72.7%	61.5%	59.3%	60.4%	0.781
Gradient Boosting	75.3%	66.0%	61.1%	63.5%	0.828

4. Backend API Design & Production Engineering

FastAPI was selected over Flask and Django due to ASGI asynchronous high-throughput concurrency, automatic OpenAPI 3.0 schema generation, and deep integration with Pydantic v2.

HTTP Method & Route	Function & Purpose	Status Code
GET /	Service discovery root metadata. Returns interactive UI for browser clients and structured JSON for REST clients.	200 OK
GET /health	Liveness & readiness probe. Checks memory residency of ML model and preprocessors.	200 / 503
POST /predict	Primary inference endpoint. Validates vitals, runs imputation, executes model, returns risk.	200 / 422
GET /metrics	Operational telemetry: Total requests served, uptime, average latency SLA, error rates.	200 OK
GET /docs	Interactive Swagger UI allowing developers and doctors to test API requests live.	200 OK

5. Top 35 Viva Voce Questions & Model Answers

Memorize these clear, concise, technical explanations for your examination:

CATEGORY A: Machine Learning & Algorithms (Q1 – Q10)

Q1. What is the core problem solved by your project?

Answer: This project provides an automated, low-latency clinical risk diagnosis system for early diabetes detection using patient vitals (Glucose, BMI, Blood Pressure, Insulin, Age), exposed via a high-performance FastAPI microservice.

Q2. Why is classification chosen over regression for this problem?

Answer: The objective is diagnostic screening—classifying whether a patient has Diabetes (1) or No Diabetes (0). Regression predicts continuous quantities, whereas binary classification produces category labels and calibrated risk probabilities.

Q3. Which algorithms did you benchmark, and which one is deployed?

Answer: We benchmarked 6 algorithms: Decision Tree, Random Forest, Logistic Regression, SVM, KNN, and Gradient Boosting. The Decision Tree model was selected as the default operational model because it achieved the highest Recall (72.2%), which minimizes dangerous False Negatives in clinical screening.

Q4. Explain what False Positives (FP) and False Negatives (FN) mean in your project.

Answer: A False Positive (Type I error) occurs when a healthy person is classified as diabetic (results in a harmless follow-up blood test). A False Negative (Type II error) occurs when a diabetic patient is mistakenly diagnosed as healthy (dangerous, as treatment is delayed). Thus, minimizing False Negatives (maximizing Recall) is critical.

Q5. What is the formula for Recall, Precision, and F1-Score?

Answer: $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$; $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$; $\text{F1-Score} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$. F1-Score is the harmonic mean balancing both metrics.

Q6. Why is Accuracy alone misleading for this dataset?

Answer: The dataset has a 65:35 class imbalance. A naive model that always predicts 'No Diabetes' would achieve 65.1% accuracy while having a 0% Recall on diabetic patients. Accuracy fails to reflect diagnostic safety on imbalanced data.

Q7. What is ROC-AUC, and what does it indicate?

Answer: Receiver Operating Characteristic (ROC) plots True Positive Rate vs False Positive Rate across all classification thresholds. AUC (Area Under Curve) measures the model's ability to discriminate between positive and negative patients. An AUC of 0.84 indicates strong ranking ability.

Q8. Why didn't you use Deep Learning / Artificial Neural Networks (ANN)?

Answer: With only 768 tabular records, Deep Learning architectures easily overfit and require huge training datasets. Tree-based and linear models generalize substantially better on small tabular datasets, train in milliseconds, and provide full explainability.

Q9. How does Random Forest work compared to Decision Trees?

Answer: A Decision Tree splits data based on Gini Impurity or Information Gain at single nodes. A Random Forest is an ensemble of multiple decision trees using Bagging (Bootstrap Aggregating) and random feature subsets to reduce variance and combat overfitting.

Q10. How do you serialize and load models in Python?

Answer: We use joblib / pickle to serialize the trained model, imputer, and scaler into .pkl binary files. During FastAPI application startup (lifespan event), models are loaded once into RAM for instant inference without disk I/O latency.

CATEGORY B: Preprocessing, EDA & Statistics (Q11 – Q18)**Q11. What are 'Disguised Zeros', and why are they significant?**

Answer: In the Pima dataset, missing laboratory observations were encoded as 0. For biological vitals like Glucose, Blood Pressure, and BMI, a value of 0 is physiologically impossible in living patients. Recognizing these as missing data (NaN) rather than genuine numeric measurements was a critical finding.

Q12. Why did you use Median Imputation instead of Mean Imputation?

Answer: Medical measurements like Serum Insulin and Skin Thickness exhibit significant right-skewness and extreme outliers. The Mean is heavily dragged by outliers, whereas the Median represents the robust central tendency of skewed biological distributions.

Q13. Why did you not drop rows containing missing/zero values?

Answer: Insulin alone had 374 missing values and Skin Thickness had 227. Dropping all zero rows would discard over 50% of the dataset, severely degrading statistical power and introducing demographic selection bias.

Q14. What is Data Leakage, and how did you prevent it?

Answer: Data Leakage occurs when information from the test dataset leaks into the training pipeline (e.g., calculating global mean/scaling parameters across the entire dataset). We strictly performed an 80/20 train-test split first; imputers and scalers were fitted ONLY on X_train and then applied to X_test.

Q15. Why is Feature Scaling necessary?

Answer: Algorithms like Logistic Regression, KNN, and SVM are sensitive to feature magnitudes because they rely on distance metrics or gradient descent. Scaling features ensures that high-magnitude variables (like Insulin 0–600) do not dominate low-magnitude variables (like Pedigree 0.05–2.5).

Q16. What is the difference between StandardScaler and RobustScaler / MinMaxScaler?

Answer: StandardScaler transforms data to zero mean and unit variance ($z\text{-score} = (x - \mu) / \sigma$). MinMaxScaler scales to a fixed range [0, 1]. RobustScaler uses the median and interquartile range (IQR), making it immune to extreme outlier distortions.

Q17. What is the Diabetes Pedigree Function?

Answer: It is a continuous score that scores genetic influence of diabetes in a patient's family history based on ancestral relationship degrees and age of onset in relatives.

Q18. How did you perform Exploratory Data Analysis (EDA)?

Answer: We generated feature distribution histograms, boxplots for outlier detection, correlation heatmaps to assess multicollinearity, and class-stratified pairplots to inspect decision boundary separability.

CATEGORY C: Backend Engineering & FastAPI (Q19 – Q27)

Q19. Why choose FastAPI over Flask or Django?

Answer: FastAPI is built on ASGI (Starlette and Uvicorn) enabling native Python async/await concurrency. It provides automatic OpenAPI/Swagger documentation, native Pydantic v2 data validation with fast Rust-based serialization, and significantly higher requests-per-second throughput than WSGI-based Flask.

Q20. What is the difference between synchronous (WSGI) and asynchronous (ASGI) servers?

Answer: WSGI servers (Flask/Django) process requests synchronously in a thread-per-request blocking model. ASGI servers (FastAPI/Uvicorn) utilize an asynchronous event loop that handles thousands of concurrent I/O-bound connections without blocking worker threads.

Q21. How does Pydantic v2 validate input data?

Answer: Pydantic enforces strict type annotations and boundary constraints at runtime. If an invalid value (e.g., negative age or string in a float field) is submitted, Pydantic intercepts the payload before hitting the ML pipeline and generates a structured HTTP 422 Unprocessable Entity error.

Q22. Explain how your API handles male patients vs female patients.

Answer: The dataset originates from female Pima Indian records where Pregnancies is a primary feature. For male patients, pregnancies are biologically 0. Our API includes a custom Pydantic validator that rejects any payload where Gender == 'male' and Pregnancies > 0 with a clear 422 error, while setting it to 0 automatically in the UI.

Q23. What is the purpose of the RequestLoggingMiddleware?

Answer: The custom middleware intercepts every HTTP request to: (1) generate a unique X-Request-ID UUID4 correlation token, (2) measure exact execution time in milliseconds, (3) log client IP and status code, and (4) inject the request ID into response headers for end-to-end tracing.

Q24. What is the function of the /health endpoint?

Answer: It acts as a readiness/liveness probe for container orchestrators (like Kubernetes or AWS ECS). It confirms whether the application is alive, whether ML model weights are loaded in RAM, and verifies that the preprocessor is functional.

Q25. What is the /metrics endpoint used for?

Answer: It exposes operational runtime telemetry including total requests processed, error rates, average latency, and system uptime, facilitating monitoring by tools like Prometheus or Datadog.

Q26. What happens if the machine learning model file is missing or corrupted?

Answer: The application's inference service implements graceful degradation. During startup, if model loading fails, the system logs a critical warning, disables the /predict route with a descriptive 503 Service Unavailable error, while keeping health and docs routes operational.

Q27. How does Content Negotiation work on the root (/) route?

Answer: The root handler inspects the HTTP Accept header. If a browser requests the page (text/html), it serves the interactive visual clinical dashboard. If an automated script or curl requests JSON (application/json), it returns structured API metadata.

CATEGORY D: Testing, Security & Deployment (Q28 – Q35)

Q28. How is the project tested automatically?

Answer: We utilize pytest with a comprehensive test suite (19 automated tests) covering Smoke tests (root, docs, health), Inference tests (positive and negative diabetes cases), Validation tests (out-of-bounds, missing fields, male pregnancy check), and Observability tests.

Q29. What is FastAPI TestClient, and does it require a running server?

Answer: FastAPI's TestClient is built on Starlette and httpx. It executes in-process HTTP requests directly against the ASGI app without needing to bind to an external network port, enabling lightning-fast unit testing in under 0.2 seconds.

Q30. What is CORS, and how is it configured?

Answer: Cross-Origin Resource Sharing (CORS) is a browser security mechanism that restricts web applications on one domain from making requests to a different domain. We configured FastAPI's CORSMiddleware to allow authorized frontend origins to query the API.

Q31. Explain the Dockerfile structure for containerization.

Answer: Our Dockerfile uses an official python:3.12-slim base image, copies requirements.txt, installs dependencies without cache to minimize image size, copies source code and model weights, exposes port 8000, and defines an ENTRYPOINT running Uvicorn.

Q32. How can this system be deployed to production cloud platforms?

Answer: The application includes a Procfile and container configurations ready for deployment on platforms like Render, AWS App Runner, Railway, or Google Cloud Run behind a reverse proxy (Nginx or Cloudflare) with auto-scaling.

Q33. How does the system handle extreme outliers during real-time inference?

Answer: Pydantic imposes hard physiological bounds (e.g., Glucose max 500 mg/dL). For extreme but valid numbers, the preprocessor uses RobustScaler which scales values based on median and IQR rather than min-max extremes.

Q34. What are the main limitations of this project?

Answer: Limitations include: (1) Dataset size of 768 female Pima Indian records may have demographic biases; (2) Binary prediction rather than multi-stage diabetes severity (e.g., Type 1 vs Type 2); (3) Absence of long-term longitudinal patient history tracking.

Q35. What is the future scope for this system?

Answer: Future enhancements include: (1) Integrating SHAP (SHapley Additive exPlanations) for local feature importance explainability; (2) Adding continuous patient time-series monitoring via wearable sensor streams; (3) Implementing OAuth2/JWT doctor authentication and HIPAA/GDPR-compliant encrypted database logging.

6. Examiner 'Trap' Questions & How to Defend

Examiners often test if you truly understand the trade-offs in your engineering decisions. Here is how to answer their trickiest questions:

Trap 1: 'Your model accuracy is ~75%. Why is it not 95% or 99%?'

Golden Defense: *'Sir/Madam, in real clinical healthcare datasets, claiming 99% accuracy on 768 records is almost always a sign of severe overfitting or data leakage. Medical data is inherently noisy and biologically variable. More importantly, our model is tuned for **Recall (72.2%)** to catch true diabetic cases. In clinical screening, a realistic 75% accuracy with high recall and zero data leakage is significantly more trustworthy and deployable than an overfitted 99% model.'*

Trap 2: 'Why did you train on a dataset of females and allow male inputs in your API?'

Golden Defense: *'The benchmark Pima dataset contains female patient vitals. To make the API medically safe and viable for real-world clinic intake, our system implements domain-specific constraint handling: for male patients, the pregnancy feature is strictly locked to 0 with automated Pydantic validation, while general metabolic vitals (Glucose, BP, Insulin, BMI, Age) remain clinically diagnostic.'*

Trap 3: 'Why didn't you use XGBoost or a Deep Neural Network?'

Golden Defense: *'For small tabular datasets (N=768), complex Deep Neural Networks suffer from high sample complexity and extreme risk of overfitting. We benchmarked 6 algorithms including ensemble methods like Random Forest and Gradient Boosting. The Decision Tree model provided the highest sensitivity (Recall) and complete algorithmic interpretability, which is required in clinical medical audits.'*

Trap 4: 'What happens if a user submits a glucose value of 10,000 mg/dL?'

Golden Defense: *'The request never reaches the machine learning model. Pydantic's strict schema layer validates that Glucose is within the plausible human range of 40 to 500 mg/dL. An immediate HTTP 422 Unprocessable Entity error is returned within 2 milliseconds with a clear error payload.'*